M1 - SMINFO2A – Patrons de conception, interactions H-M

- Rapport -
CC Patrons de conception

Jeu 2D d’occupation maximale d’espace

Membres du groupe D :

- Khanie TSITANA
- Danylo SUKACH
- Bessy Carelle IGIRANEZA
- Remy DEFARIA

Sommaire:

- Fonctionnement de l’application
- Les patterns utilisés :
    - MVC
    - State
    - Observer
    - Strategy
    - Command

- Attribution des tâches
- Annexes (Captures d’écran de l’appli)

A - Fonctionnement de l’application

Indication rapide de toutes les fonctionnalités de l’application :

- **De base :**

Le jeu est en mode 1 joueur de base, des formes rouges sont générées aléatoirement sur la fenêtre et qui disparaissent au fil du temps pour plus de difficulté. Le but du joueur est de remplir le plus d'espace vide sans toucher les formes rouges avec ses formes.

- **bouton Cercle :**

Permet de se mettre en mode “création de cercle”, premier clic pour définir le centre et le deuxième pour valider le cercle avec le rayon.

- **bouton Rectangle :**

Permet de se mettre en mode “création de rectangle”, premier clic pour définir le point en gauche et le deuxième pour définir le point en bas à droite et valider la création du rectangle.

- **bouton Undo :**

Permet de revenir en arrière sur l’action précédente.

- **bouton Redo :**

Permet de revenir en avant sur une action où on est revenu en arrière.

- **bouton Valider :**

Permet de valider ses formes dans le mode 1 Joueur et le mode 2 Joueur.

<u>(ne pas hésiter à agrandir la fenêtre si l’on ne voit pas les bouton ci-dessous)</u>

- **bouton Quitter :**

Permet de quitter l’application.

- **bouton 2J :**

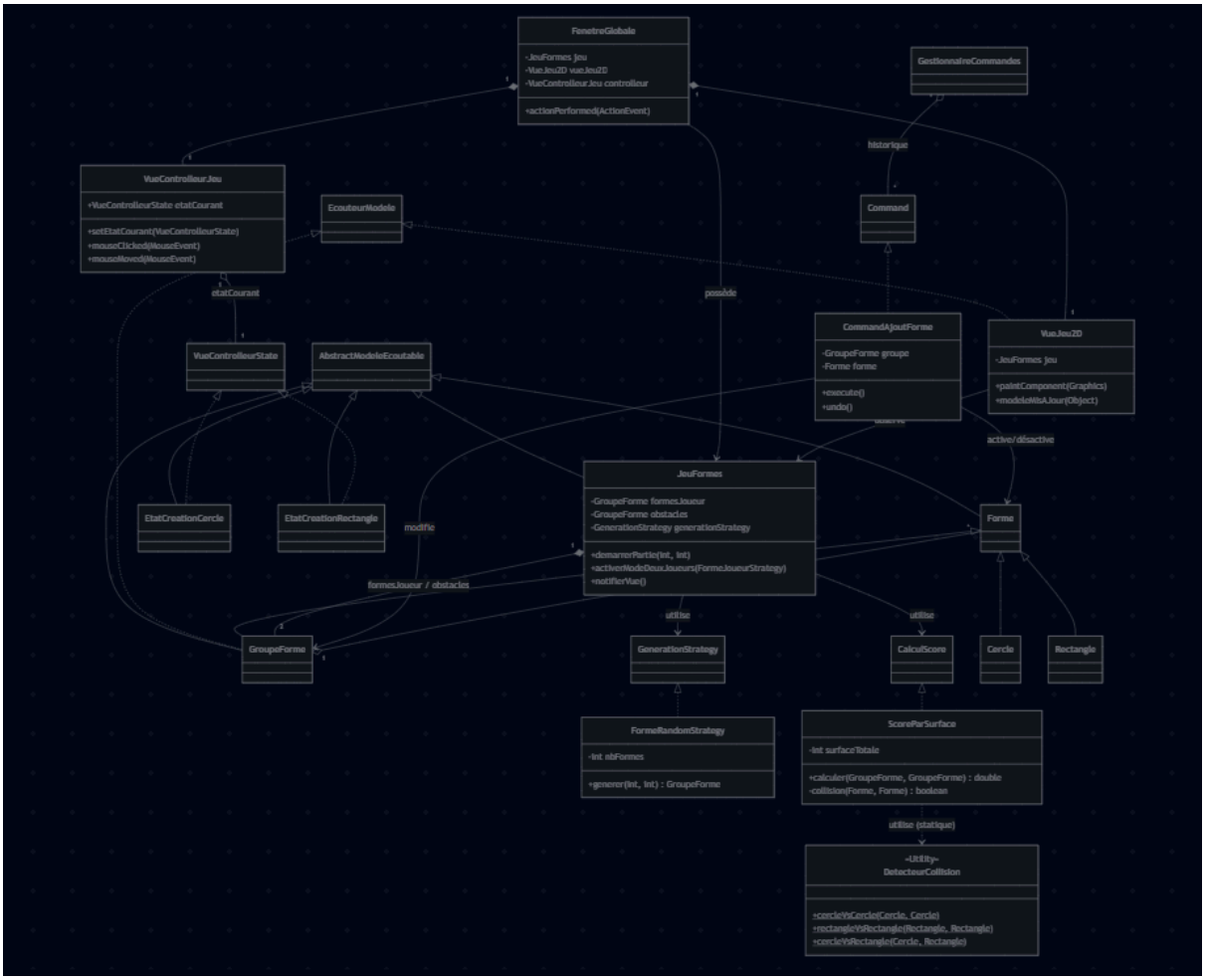
Permet d'accéder au mode 2 Joueurs. Toutes les informations concernant le jeu sont indiquées directement sur le jeu.

B - Les patterns utilisées :

Pour réaliser notre projet, nous avons utilisé les patterns suivants:

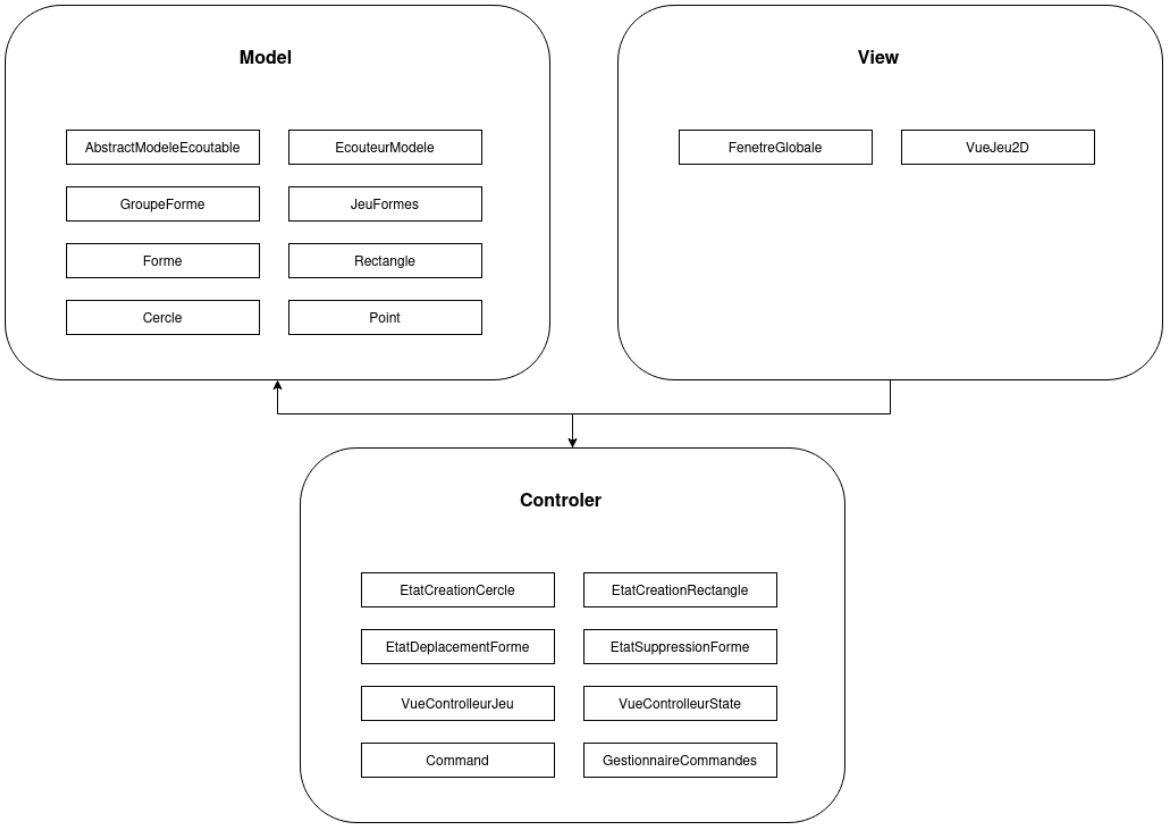
- MVC
- State
- Observer
- Strategy
- Command

Diagramme de classe non exhaustif complet du projet



I - Pattern MVC

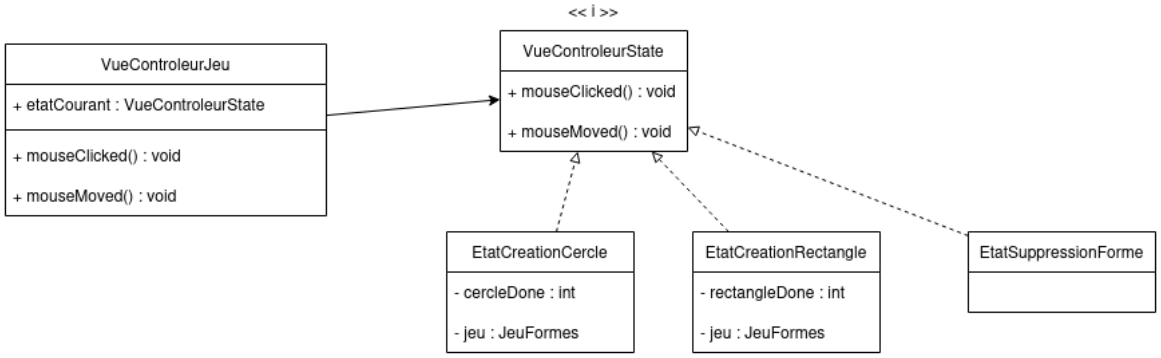
Notre projet suit la structure MVC et les fichiers sont subdivisés dans les dossiers Model, View et Controler, selon leur fonctionnalités. Le diagramme suivant montre la structure de notre projet avec les noms de classe qui se trouvent dans Model, View et Controler.
Tout ce qui est visuel se trouve dans **View**, les actions à effectuer après une interaction dans View sont traitées dans **Controler** et ce dernier utilise les Objets de **Model**.



II - Pattern State

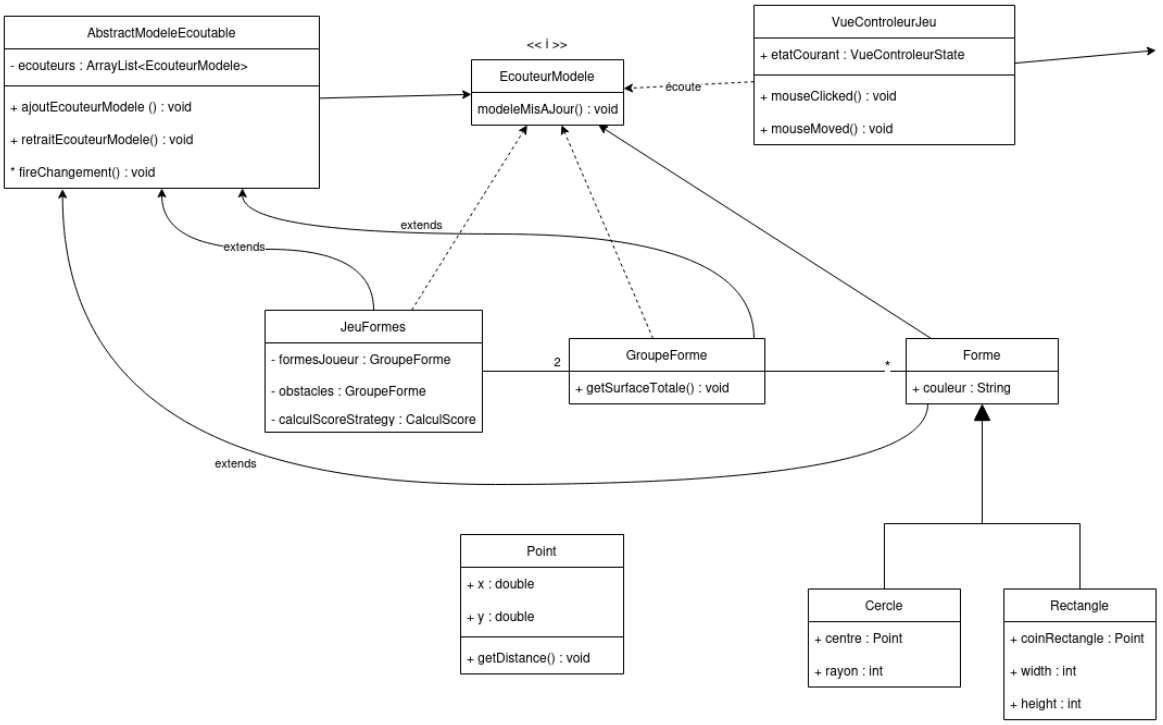
Pour pouvoir changer d'état entre les différentes options sur la fenêtre du jeu : Créer Cercle, Créer Rectangle, Supprimer forme (à implémenter) etc.., on a utilisé le pattern State.
On a un état courant sur ***VueControleurJeu*** qui est soit ***EtatCreationCercle*** soit, ***EtatCreationRectangle***, on change d'état lorsqu'on clique sur les boutons respectifs sur l'interface globale.
L'interface va ainsi pouvoir recevoir la position de la souris, capturer les clicks de souris et traiter ces informations selon l'état courant. Il va définir un centre et dessiner un cercle si on est dans l'état création cercle et définir un coin et dessiner un rectangle si on est dans l'état création Rectangle.
Ce pattern est étroitement lié avec le pattern Observer. Grâce aux deux patterns liés, on peut dessiner les formes en temps réel (on voit l'évolution de la forme quand on bouge la souris), grâce à la classe ***VueJeu2D*** qui écoute la position de la souris et les différents éléments de ***GroupeForme***, à chaque mouvement et en considérant l'état courant.

La figure ci-dessus montre le diagramme de classe du Pattern State, séparé du reste du diagramme du système.



III - Pattern Observer

Le pattern Observateur permet de lier les différentes classes de notre projet et de propager les changements entre elles. *La figure ci-dessous montre le diagramme de classe du pattern observer, légèrement mélangé avec d'autres Pattern.*



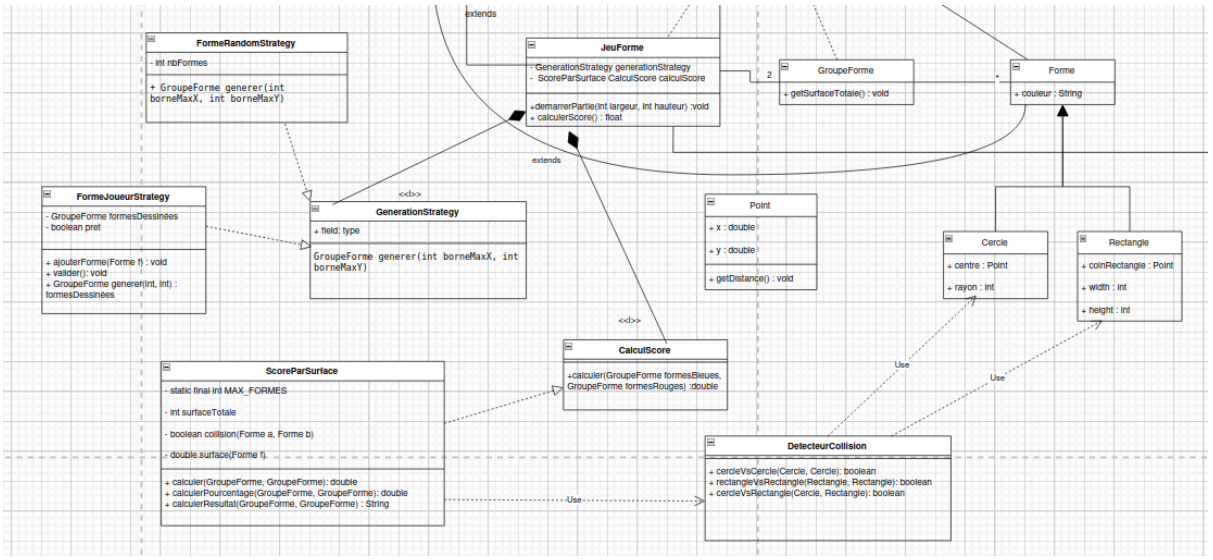
IV - Pattern Strategy

L’architecture du projet repose sur le **Pattern Strategy**.Il permet de dissocier la logique principale du jeu des différentes façons de gérer le score et la génération.
Cela rend le code plus flexible et facile à faire évoluer sans modifier les classes de base.

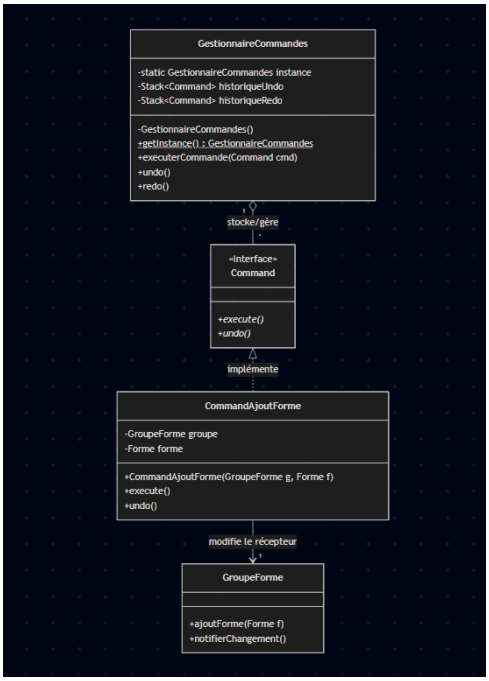
-Stratégie de génération des formes (“GenerationStrategy)”:
Cette interface permet de définir comment les obstacles apparaissent sur le plateau.
FormeRandomStrategy : L'ordinateur génère des formes rouges de manière aléatoire (utilisé pour le monoJoueur).
FormeJoueurStrategy : Permet à un premier joueur de dessiner manuellement les zones rouges pour piéger le second joueur (mode 2 joueurs).

-Stratégie de calcul du score (“CalculScore”):
Cette interface gère la validation des performances du joueur.
ScoreParSurface : C'est la classe principale pour le calcul du score. Il utilise la classe “DetecteurCollision” pour vérifier si les formes bleues touchent les zones rouges.
Il calcule deux types de résultats :
-Le score de précision : Nombre de formes valides par rapport au maximum autorisé (4 formes).
-Le score de remplissage de la surface : Pourcentage de la surface libre réellement occupée par le joueur.

Grâce à cette structure de code, la classe “JeuFormes” n'a pas besoin de savoir comment on calcule ou comment on génère. Il lui suffit d'appeler les méthodes génériques. On pourrait ainsi ajouter une autre stratégie de jeu IA ou faire une partie de 10 jeux en les conservant dans un fichier sans casser le reste du code.



V - Pattern Command



L'interface Command définit le contrat execute et undo que toutes les actions du jeu doivent suivre. CommandAjoutForme est la commande concrète, elle connaît le "Récepteur" GroupeForme et l'action à effectuer. GestionnaireCommandes c'est lui qui gère l'historique avec deux piles de Stack pour mémoriser les actions et permettre de revenir en arrière ou en avant. La figure ici présente, montre le diagramme de classe du Pattern Command plus précisément.

C - Attribution des tâches

Les contributions pour chacun des pattern :

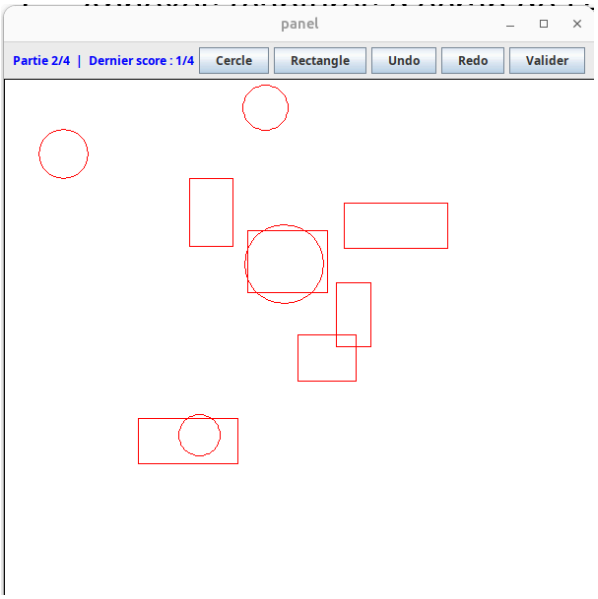
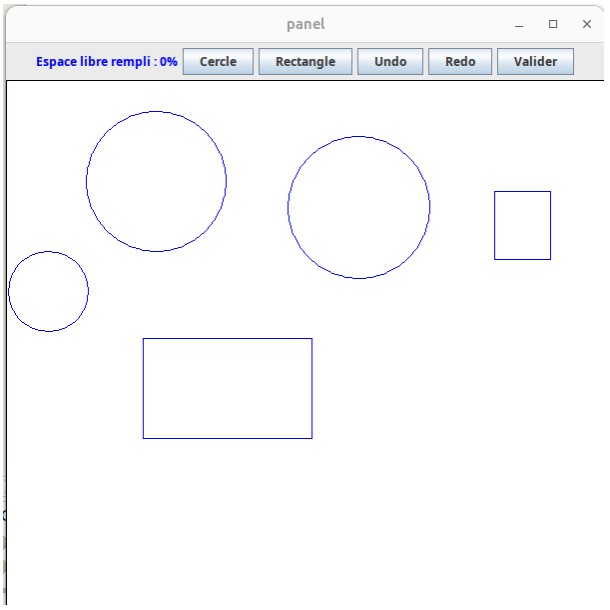
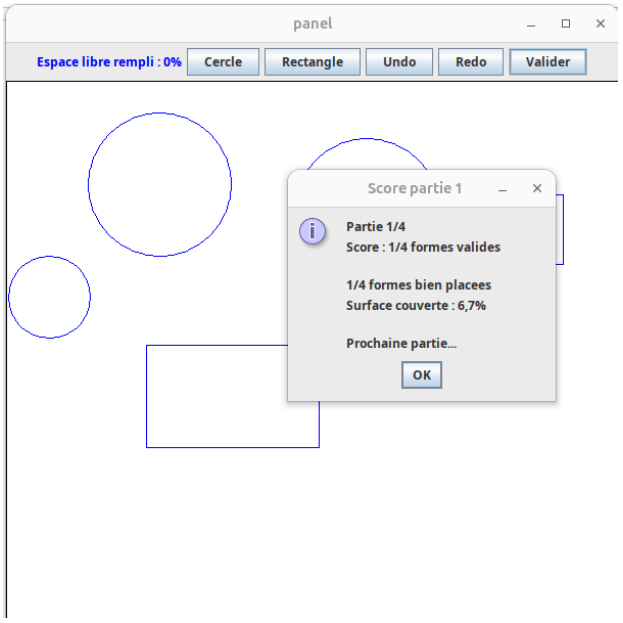
- **Pattern Observer :** Danylo et Khanie
- **Pattern State :** Danylo et Khanie
- **Pattern Strategy :** Rémi et Bessy
- **Pattern Command :** Rémi

Les noms des classes que chacun a créé :
DANYLO : AbstractModeleEcoutable, EcouteurModele, compilation du fichier .jar final.

KHANIE : GroupeForme, Forme, Cercle, Rectangle, Point, EtatCreationCercle, EtatCreationRectangle, VueControlleurJeu, VueControlleurState, FenetreGlobale, VueJeu2D.

BESSY : GenerationStrategy, FormeRandomStrategy, calculScore, ScoreParSurface, JeuFormes, Stratégie du jeu (monojoueur) ,DetecteurCollision et calcul du score.

RÉMI : Command, CommandAjoutForme, GestionnaireCommandes, FormeJoueurStrategy

Pour le **pattern MVC**, chacun a essayé de subdiviser ses classes respectives selon leur fonctionnalités dans Model, View ou Controler.

D - Annexes (captures d'écran de l'appli)